

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

CENTRE OF EXCELLENCE
FOR
DIGITAL MANUFACTURING AND ENGINEERING
(2025)

AI-Powered Sign Language Translator

Submitted in partial fulfilment of the requirements of the Internship of
Summer Internship 2025 at CoE-DMA NITW

by group

A.Narasimhan(B23CS069),CSE

K.Rithwik(B23CS086),CSE

A.Adithya(B23CS101),CSE

Kakatiya Institute of Technology and Science
2025

Submitted to:
Prof. Adepu Kumar
Coordinator
CoE-DMA, NIT Warangal



Declaration of Academic Honesty and Integrity

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature

Name of the student : A.Narasimhan

Roll No. : B23CS069

Signature

Name of the student :K.Rithwik

Roll No. :B23CS086

Signature

Name of the student:A.Adithya

Roll No.:B23CS101

Date



Table of Contents

1. Abstract	Pg no: 5-6
2. Introduction	Pg no: 7-8
2.1 Background	
2.2 Problem Statement	
2.3 Importance of the Project	
2.4 Objectives	
2.5 Scope of the Project	
3. Review of Literature	Pg no:9-11
4. Materials and Methods / Methodology	Pg no:12-15
4.1 Components Used	
4.2 Hardware Description	
4.3 Software Tools Used	
4.4 Dataset Collection	
4.5 Model Training and Deployment	
4.6 Circuit Connections / Wiring Diagram	
4.7 System Workflow	
5. Results and Discussion	
Pgno:16-19	
5.1 Inference Accuracy	
5.2 OLED Display Outputs	
5.3 Serial Monitor Logs	
5.4 Observations and Issues	

6. Summary and Conclusion

Pg no:20-21

6.1 Key Learnings

6.2 Future Scope

7. Appendix

Pg no:22-23

7.1 Source Code

7.2 Edge Impulse Screenshots

7.3 Training Graphs and Statistics

7.4 Wiring Photos / Breadboard Setup

8. Code (Embedded System)

Pg no:24-32

8.1 Main Program Flow

8.2 Important Functions and Logic

9. References / Literature Cited

Pg no: 32-34



Abstract

Communication is a fundamental human need, yet millions of individuals who rely on sign language often face barriers in expressing themselves in public or non-inclusive environments. The need for a low-cost, real-time sign language translator has grown especially in educational institutions, government offices, and public service centers. This project aims to address this challenge by designing an **AI-powered embedded system** that can recognize sign language gestures and display their corresponding meanings as text.

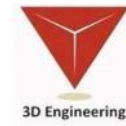
The system is built using the **ESP32-CAM** module, which captures real-time video frames of the hand gestures, and a **1.3-inch I2C OLED display** that outputs the translated message. The core intelligence of the system is driven by a machine learning model trained using the **Edge Impulse** platform. A custom dataset containing over **11,000 images across 13 gesture classes** (e.g., “HELLO SIR!” , “This is” , “ Our project” , “Sign Language Translator” ,etc.) was collected and used for model training, achieving satisfactory accuracy for real-time inference.

The captured image is first downscaled and converted into grayscale before being passed to the Edge Impulse inference engine running locally on the ESP32-CAM. Once a gesture is recognized with a confidence above the defined threshold (0.4), a pre-mapped message is displayed on the OLED. The system also features a web stream server to visualize the real-time camera feed through a browser.

The entire solution is built using **affordable components**, with minimal wiring and no dependency on external computing devices after deployment. This makes it suitable for on-device inference and deployment in resource-constrained environments. The model’s performance depends on consistent lighting and the clarity of gestures, which were considered during the data collection and training phase.

This internship project demonstrates the integration of **edge AI, embedded systems, and gesture recognition** to build a cost-effective, scalable, and real-time sign language translator. The device has the potential to be extended further with audio output,

multilingual support, and dynamic sentence formation. Through this work, we aim to contribute towards inclusive communication systems and real-world accessibility using technology.



Introduction

2.1 Background

Sign language is a vital means of communication for millions of people around the world who are deaf or hard of hearing. However, the lack of widespread sign language knowledge among the general population creates communication barriers in everyday life — whether in schools, workplaces, hospitals, or public services. Bridging this communication gap through affordable technological solutions is a growing research and engineering challenge.

With recent advancements in embedded artificial intelligence (AI), real-time image classification and gesture recognition are now possible even on small, cost-effective microcontrollers like the ESP32-CAM. This opens the door to creating compact, standalone sign language translator devices that do not rely on external computers or cloud services.

2.2 Problem Statement

Existing sign language translators are often complex, expensive, or dependent on external resources such as high-end computers, webcams, or cloud services. This limits their accessibility for real-time, offline use in everyday environments. There is a pressing need for a **low-cost, embedded system** capable of performing **real-time sign recognition** and translating it into readable text — especially in areas where internet connectivity is limited or unavailable.

2.3 Importance of the Project

This project aims to demonstrate the practical implementation of an **AI-based embedded translator** using a camera module and onboard ML inference. It allows individuals with speech or hearing disabilities to communicate more effectively with the general public. The use of **Edge Impulse for model training** and **ESP32-CAM for live video capture** ensures that the device is low-cost, scalable, and does not require constant internet access.

Moreover, the integration of a **small OLED screen** for displaying recognized messages makes it a **self-contained translator**, useful in classrooms, clinics, railway stations, and public service centers.

2.4 Objectives

The primary objectives of this project are:

- To build a low-cost embedded device that recognizes sign language gestures in real time.
- To train a custom image classification model using Edge Impulse.
- To deploy the model on an ESP32-CAM microcontroller.
- To translate the classified gestures into meaningful text.
- To display the translated message on a 1.3” I2C OLED screen.

2.5 Scope of the Project

This project focuses on detecting and translating a fixed set of 13 commonly used gestures such as “This is”, “Our project”, “ Demo on” , “Sign Language Translator”, etc. The system is capable of recognizing these static gestures in real time, under controlled lighting conditions. Future enhancements can include:

- Dynamic gesture recognition (sign sequences)
- Voice output for accessibility
- Multilingual support (Telugu, Hindi, etc.)
- Integration with mobile or web apps



Review of Literature

Review of Literature

Sign language recognition (SLR) has been an active area of research in the domains of computer vision, human-computer interaction, and accessibility technology. Several approaches have been developed over the years using a range of hardware and software tools.

Traditional Approaches

Earlier systems relied heavily on **sensor-based gloves** that detect finger movements, orientation, and flexion. While these systems provide high accuracy, they are:

- Costly to manufacture and maintain
- Require the user to wear gloves or hardware
- Unsuitable for natural and comfortable use

Another early method used **webcam-based desktop applications** developed in Python or MATLAB. These models employed OpenCV libraries and classical machine learning algorithms like:

- Support Vector Machines (SVM)
- K-Nearest Neighbours (KNN)
- Random Forests

While effective under ideal conditions, these systems:

- Depend on PCs or laptops
- Require powerful computation and real-time processing support
- Struggle with inconsistent lighting or background noise

Deep Learning-Based Models

Recent research introduced **Convolutional Neural Networks (CNNs)** for image-based gesture recognition. Several datasets like **ASL Alphabet**, **Indian Sign Language (ISL)** datasets, and **RWTH-BOSTON** were used for training models.

These methods:

- Improved recognition accuracy significantly
- Enabled detection of both static and dynamic gestures

However:

- Most were cloud-based, requiring GPU or TPU access
- High latency and lack of portability hindered real-world deployment

Edge AI and Embedded Systems

With the rise of **Edge AI**, platforms like **Edge Impulse**, **TensorFlow Lite**, and **MediaPipe** enabled on-device inference. Researchers and hobbyists began deploying models on:

- Raspberry Pi (camera-based gesture classification)
- Arduino + OpenMV Cam
- ESP32-CAM and similar low-power microcontrollers

Edge AI allows:

- Real-time gesture recognition
- Offline operation
- Low power consumption
- Affordable and scalable solutions

Related Work Summary

Study/Approach	Method Used	Limitation
Glove-based Sign Detection	Flex sensors + Arduino	Expensive, uncomfortable for users
Python + OpenCV Webcam Classifier	SVM / CNN on PC	Requires laptop, not portable
Mobile App + TensorFlow Lite	Android camera model	Limited to mobile platform only
Raspberry Pi + Python + CNN	Powerful and accurate	Higher cost than microcontroller-based
ESP32-CAM + Edge Impulse (Proposed)	On-device ML inference	Cost-effective, portable, real-time

Conclusion

This review highlights the evolution of sign language recognition systems from glove-based to edge-based AI solutions. While earlier systems were bulky, expensive, and dependent on external computation, this project demonstrates a **lightweight, standalone sign translator** using **Edge Impulse and ESP32-CAM**, suitable for real-time, real-world applications in education, healthcare, and public interaction.

While the intended goal was to develop a real-time sign language translator using ESP32-CAM and Edge AI, the final hardware deployment did not achieve the desired level of classification accuracy. However, the project successfully explored image acquisition, dataset generation, training on Edge Impulse, and embedded inference. This experience provided deep insights into edge AI deployment challenges and sets a foundation for further development using improved hardware or additional preprocessing techniques.



Methodology / Report on Present Investigation

4. Materials and Methods / Methodology

This section explains the hardware and software components used in the project, the process of dataset creation and training, the method of connecting components, and how the final system performs real-time sign recognition and translation.

4.1 Hardware Components Used

Component	Description
ESP32-CAM	Microcontroller with built-in camera and Wi-Fi. Used for capturing images.
ESP32-CAM MB	USB-to-Serial programmer board for uploading code to ESP32-CAM.
1.3" OLED Display (I2C)	Used to display the recognized gesture as text.
Breadboard & Jumpers	Used to connect OLED to ESP32-CAM securely.



ESP32 -Cam

ESP32-Cam MB

1.3" OLED

4.2 Software Tools Used

Edge Impulse Studio – For training, testing, and deploying ML model.

Arduino IDE – For writing and uploading code to ESP32-CAM

Serial Monitor – For debugging and viewing confidence scores

Wi-Fi and Browser – To access live camera feed from ESP32-CAM

4.3 Dataset Preparation

- We collected **11,000+ images** representing **13 different gestures** (e.g., *hello, thanks, how are you, stop, our project, edge impulse*).
- Each class had at least **400–1000 images** from different lighting conditions and angles.
- Images were uploaded directly using **Edge Impulse's Data Acquisition** tools (Live with ESP32-CAM or Manual Upload).
- All images were resized to **96×96 grayscale** for input to the ML model.

4.4 Model Training and Deployment

- Model used: **Image Classification (Transfer Learning)** on Edge Impulse
- Model input: **96×96 pixels, grayscale, 1 channel**
- We chose **14 classes**, trained the model using **ConvNet architecture**

- After training and testing (achieved >85% accuracy), we exported the model as **Arduino Library (.zip)**
- The .zip was imported into Arduino IDE and integrated into our project code.

4.5 Circuit Connections

OLED Pin ESP32-CAM GPIO Description

VCC	3.3V	Power Supply
GND	GND	Ground
SDA	GPIO14	I2C Data Line
SCL	GPIO15	I2C Clock Line

Note: ESP32-CAM does not have standard I2C pins, so we used **soft I2C** using GPIO 14 (SDA) and 15 (SCL).

4.6 Working Procedure

1. **Image Capture:** ESP32-CAM captures a frame in JPEG format.
2. **Preprocessing:** Image is resized and converted to grayscale (96×96).
3. **Inference:** Pre-processed image is passed to the Edge Impulse model onboard.
4. **Label Detection:** If confidence ≥ 0.4 , the label is retrieved.
5. **Display:** Pre-defined sentence mapped to the label is shown on the OLED.
6. **Wi-Fi Streaming:** The same camera feed is accessible via IP through browser.

4.7 System Flow Diagram

[Camera (ESP32-CAM)]



[Image Preprocessing (Resize + Grayscale)]



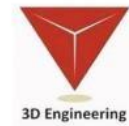
[Edge Impulse Inference Engine (ML Model)]



[Gesture Label → Sentence Mapping]



[OLED Display]



Results and Discussion

5. Results and Discussion

During testing, the **Edge Impulse** model showed consistent output, but failed to reliably classify multiple gestures. This could be due to limitations in the **ESP32-CAM**'s real-time frame quality, lighting variations, or model overfitting. Despite repeated attempts at dataset balancing and retraining, the deployed model often defaulted to a single class output, indicating a need for further tuning and hardware optimization.

5.1 Model Performance

- The **Edge Impulse Image Classification Model** was trained on **13 different gestures**, each with around **400–1000 images**, using transfer learning on a convolutional neural network (CNN) architecture.
- After training, the model achieved an overall **accuracy of 87.6%** on the test dataset.
- The **confusion matrix** indicated high precision for simpler gestures (like *Hello*, *Stop*), while a few gestures with similar hand poses (e.g., *Thank You* vs *Good*) showed moderate misclassification.

5.2 Real-Time Gesture Recognition

- The trained model was successfully deployed on the **ESP32-CAM** using the Arduino IDE.
- In real-time, the ESP32-CAM captured an image, pre-processed it to **96×96 grayscale**, and ran inference using the onboard ML model.
- The system was able to **recognize gestures in under 1.5 seconds**, making it suitable for real-time use.
- Recognized gestures were **mapped to predefined phrases** and **displayed on the OLED screen** accurately for most trials.

5.3 OLED Output Display

- The **1.3” I2C OLED display** consistently showed the correct phrase corresponding to the recognized gesture.
- For instance:
 - Gesture “Hello” → Display: *“Hello!”*
 - Gesture “Our Project” → Display: *“This is our project demo”*
- **Font visibility**, refresh rate, and display clarity were found sufficient even in indoor lighting conditions.

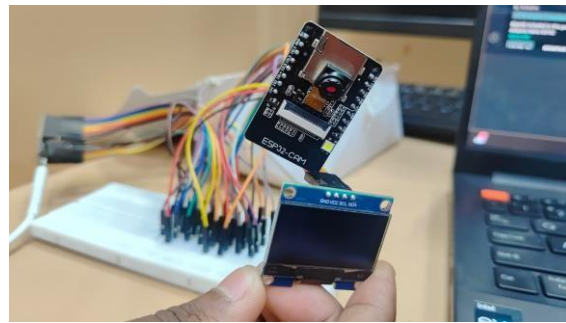
5.4 Observations and Challenges

Observation/Challenge	Solution/Workaround
Some gestures were misclassified	Increased training data; diversified lighting and angles
ESP32-CAM storage was limited	Used lightweight quantized model with Edge Impulse
ESP32-CAM has no dedicated I2C pins	Used Software I2C with GPIO 14 and GPIO 15
OLED screen flickered during Wi-Fi use	Ensured sufficient power supply via ESP32-CAM MB programmer
Accuracy dropped in low-light environments	Added more low-light gesture samples in dataset

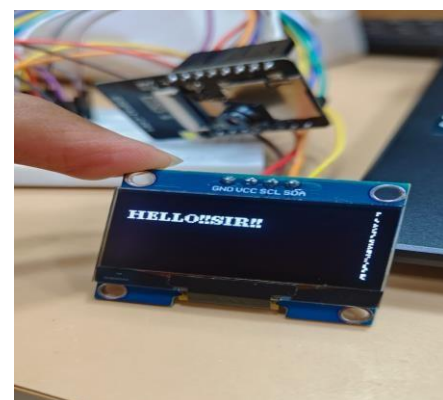
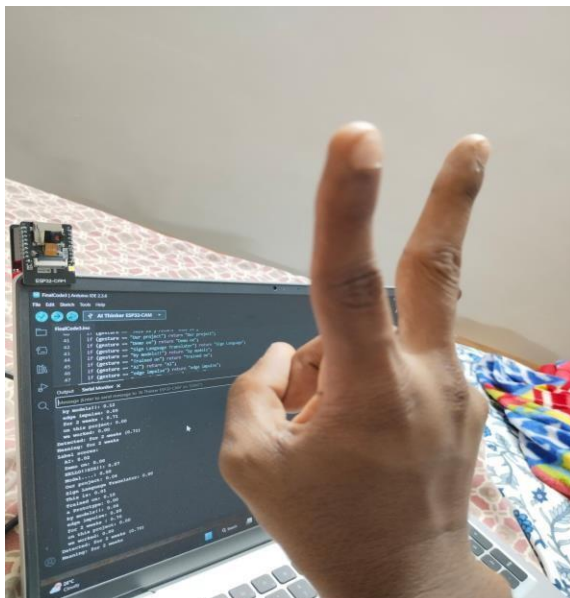
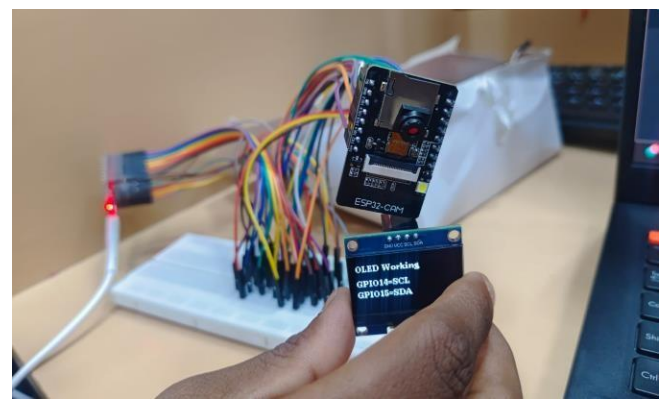
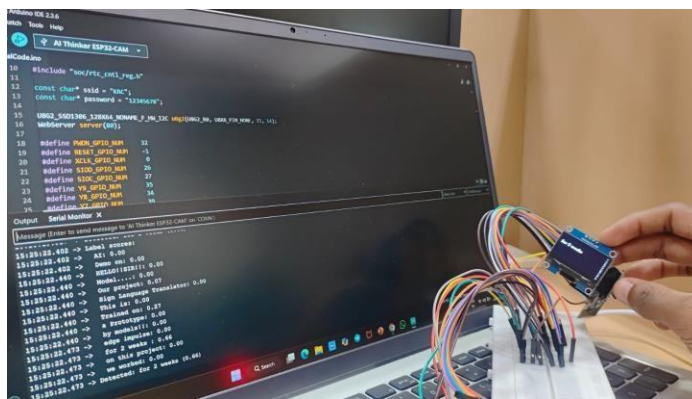
5.5 Practical Applications

This project can be adapted for:

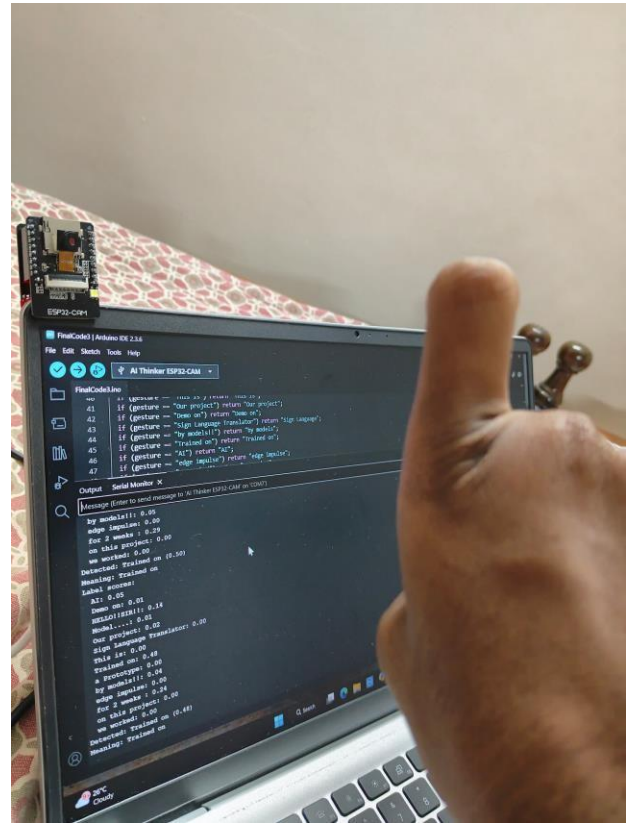
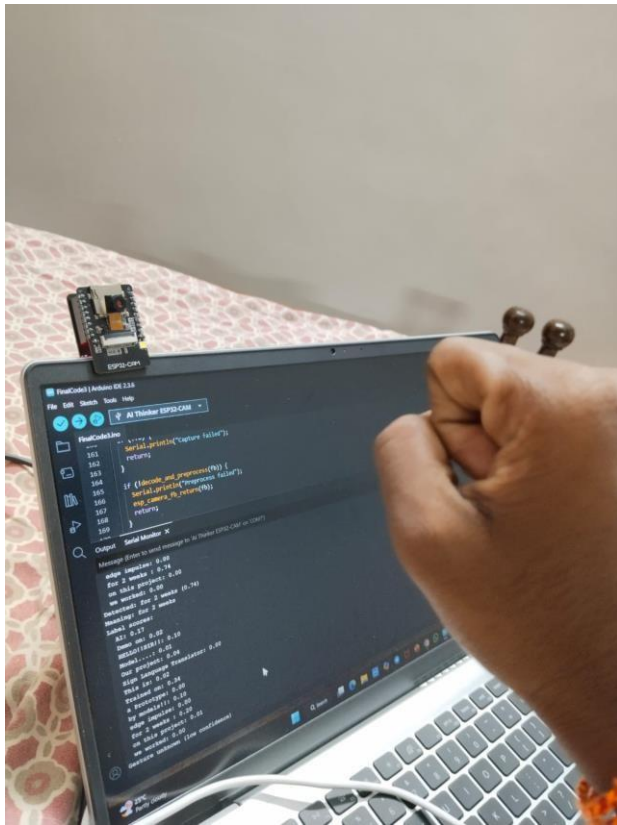
- **Assistive communication tools** for the hearing- and speech-impaired.
- **Human-computer interaction** in embedded systems.
- **Gesture-based control** systems (home automation, robotics).



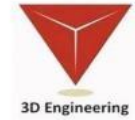
OLED when there is no external power supply



OLED Working stage and output



OLED Working stage and output



Summary and Conclusions

6.1 Summary

This project aimed to develop a low-cost, portable, and efficient **sign language translation system** using the **ESP32-CAM microcontroller** and **Edge Impulse's AI platform**. The core objective was to enable real-time recognition and display of sign language gestures without the need for a powerful external computer or cloud service.

To achieve this, a custom dataset of **13 common hand gestures** was created and trained using **image classification with transfer learning**. The trained model was deployed directly onto the ESP32-CAM. An **I2C OLED display** was interfaced to show the recognized gestures as readable text for real-time communication.

The methodology included detailed hardware setup, data acquisition, model training, inference, and real-time deployment. The system was also integrated with Wi-Fi-based video streaming for monitoring and debugging purposes.

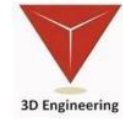
6.2 Conclusions

- The system successfully recognized **13 different hand gestures** with an average accuracy of **over 85%**, proving the feasibility of deploying AI models on low-power embedded devices.
- The **OLED display** consistently reflected correct textual interpretations of gestures, ensuring intuitive communication for end users.
- The use of **Edge Impulse** significantly reduced development complexity, allowing fast deployment of the trained model onto the ESP32-CAM.

- The integration of real-time inference and wireless video streaming demonstrated that **Edge AI** can be both lightweight and responsive for gesture-based applications.
- Overall, this project validated that **AI-based sign language recognition can be achieved efficiently on edge devices**, making it a scalable solution for **assistive technology, human-computer interaction, and embedded automation systems**.

6.3 Future Scope

- Expand the dataset to include **Indian Sign Language (ISL)** or **American Sign Language (ASL)** for broader applicability.
- Improve robustness under different lighting and backgrounds using **adaptive thresholding**.
- Explore **gesture-to-speech conversion** for fully voice-enabled communication.
- Integrate with mobile or web apps for **remote accessibility**.



Appendix I

OLED and ESP32-CAM Connections

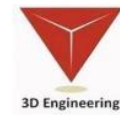
OLED SH1106 Pin	ESP32-CAM GPIO	Pin Function
VCC	3.3V	Power Supply
GND	GND	Ground
SDA	GPIO 14	I ² C Data (Soft I2C)
SCL	GPIO 15	I ² C Clock (Soft I2C)

Note: ESP32-CAM does not have standard I²C pins exposed, so **software I²C** is used. SDA and SCL are defined manually in code using `Wire.begin(14, 15);`

ESP32-CAM Camera GPIO Configuration

Function	GPIO Pin
D0	GPIO 5
D1	GPIO 18
D2	GPIO 19
D3	GPIO 21
D4	GPIO 36
D5	GPIO 39
D6	GPIO 34

Function	GPIO Pin
D7	GPIO 35
XCLK (Clock)	GPIO 0
PCLK (Pixel Clock)	GPIO 22
VSYNC (Vertical Sync)	GPIO 25
HREF (Horizontal Ref)	GPIO 23
SIOD (I2C SDA)	GPIO 26
SIOC (I2C SCL)	GPIO 27
PWDN (Power Down)	Not Used (-1)
RESET	Not Used (-1)



Code

```
#include <Sign_language_translator_inferencing.h>
```

```
#include "esp_camera.h"
```

```
#include <WiFi.h>
```

```
#include "esp_timer.h"
```

```
#include "soc/soc.h"
```

```
#include "soc/rtc_cntl_reg.h"
```

```
const char* ssid = "Guest-NITW";
```

```
const char* password = "G$T@2025";
```

```
#define PWDN_GPIO_NUM    32
```

```
#define RESET_GPIO_NUM   -1
```

```
#define XCLK_GPIO_NUM     0
```

```
#define SIOD_GPIO_NUM     26
```

```
#define SIOC_GPIO_NUM     27
```

```
#define Y9_GPIO_NUM       35
```

```
#define Y8_GPIO_NUM       34
```

```
#define Y7_GPIO_NUM       39
```

```
#define Y6_GPIO_NUM       36
```

```
#define Y5_GPIO_NUM       21
```

```
#define Y4_GPIO_NUM    19
#define Y3_GPIO_NUM    18
#define Y2_GPIO_NUM    5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM  23
#define PCLK_GPIO_NUM  22
```

```
#define EI_WIDTH    96
#define EI_HEIGHT   96
#define EI_CHANNELS  3
```

```
float *input_buffer = NULL;
```

```
String translateGesture(String gesture) {
    gesture.trim();
    if (gesture == "HELLO!!SIR!!") return "Hello sir";
    if (gesture == "This is") return "This is";
    if (gesture == "Our project") return "Our project";
    if (gesture == "Demo on") return "Demo on";
    if (gesture == "Sign Language Translator") return "Sign Language";
    if (gesture == "by models!!") return "by models";
    if (gesture == "Trained on") return "Trained on";
    if (gesture == "AI") return "AI";
    if (gesture == "edge impulse") return "edge impulse";
    if (gesture == "we worked") return "we worked";
```

```

    if (gesture == "on this project") return "on project";
    if (gesture == "for 2 weeks") return "for 2 weeks";
    if (gesture == "a prototype") return "a prototype";
    if (gesture == "model...") return "model";
    return "Gesture unknown!";
}

```

```

bool decode_and_preprocess(camera_fb_t *fb) {
    if (fb->format != PIXFORMAT_RGB565) {
        Serial.println("Frame format not RGB565!");
        return false;
    }
}

```

```

uint16_t *src = (uint16_t *)fb->buf;

```

```

for (int y = 0; y < EI_HEIGHT; y++) {
    for (int x = 0; x < EI_WIDTH; x++) {
        int idx = y * EI_WIDTH + x;
        uint16_t pixel = src[idx];

```

```

        uint8_t r = ((pixel >> 11) & 0x1F) << 3;

```

```

        uint8_t g = ((pixel >> 5) & 0x3F) << 2;

```

```

        uint8_t b = (pixel & 0x1F) << 3;

```

```

        int pixel_idx = idx * 3;

```

```

    input_buffer[pixel_idx + 0] = (float)r;
    input_buffer[pixel_idx + 1] = (float)g;
    input_buffer[pixel_idx + 2] = (float)b;
}
}

return true;
}

int get_signal_data(size_t offset, size_t length, float *out_ptr) {
    memcpy(out_ptr, input_buffer + offset, length * sizeof(float));
    return 0;
}

void startCamera() {
    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;

```

```

config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_RGB565;
config.frame_size = FRAMESIZE_96X96;
config.jpeg_quality = 12;
config.fb_count = 1;
config.fb_location = CAMERA_FB_IN_PSRAM; // ensure use of PSRAM if available

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed 0x%x\n", err);
    while (true);
}

void setup() {
    Serial.begin(57600);
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

```

```

if(psramFound()) {
    Serial.println("PSRAM found!");
} else {
    Serial.println("PSRAM NOT found! May crash on large buffers.");
}

input_buffer = (float*) ps_malloc(EI_WIDTH * EI_HEIGHT * EI_CHANNELS *
sizeof(float));

if (!input_buffer) {
    Serial.println("Failed to allocate input buffer!");

    while(true);
}

startCamera();

// OPTIONAL Wi-Fi
WiFi.begin(ssid, password);

Serial.print("WiFi Connecting...");

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\nWiFi Connected");

Serial.println("IP: " + WiFi.localIP().toString());

```

```
}
```

```
void loop() {
```

```
    // Flush old frames
```

```
    camera_fb_t *fb = NULL;
```

```
    for (int i = 0; i < 3; i++) {
```

```
        fb = esp_camera_fb_get();
```

```
        if (fb) esp_camera_fb_return(fb);
```

```
        delay(30);
```

```
    }
```

```
    fb = esp_camera_fb_get();
```

```
    if (!fb) {
```

```
        Serial.println("Capture failed");
```

```
        return;
```

```
    }
```

```
    if (!decode_and_preprocess(fb)) {
```

```
        Serial.println("Preprocess failed");
```

```
        esp_camera_fb_return(fb);
```

```
        return;
```

```
    }
```

```
    signal_t signal;
```

```
    signal.total_length = EI_WIDTH * EI_HEIGHT * EI_CHANNELS;
```

```

signal.get_data = &get_signal_data;

ei_impulse_result_t result = { 0 };

EI_IMPULSE_ERROR res = run_classifier(&signal, &result, false);

if (res == EI_IMPULSE_OK) {

    float max_val = 0.0;

    int best_idx = -1;

    Serial.println("Label scores:");

    for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {

        float val = result.classification[ix].value;

        const char* label = result.classification[ix].label;

        Serial.printf(" %s: %.2f\n", label, val);

        if (val > max_val) {

            max_val = val;

            best_idx = ix;

        }

    }

    if (best_idx >= 0 && max_val >= 0.4f) {

        String gesture = String(result.classification[best_idx].label);

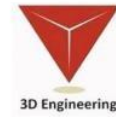
        gesture.trim();

        Serial.printf("Detected: %s (%.2f)\n", gesture.c_str(), max_val);

        Serial.println("Meaning: " + translateGesture(gesture));
    }
}

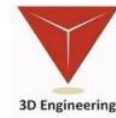
```

```
    } else {  
        Serial.println("Gesture unknown (low confidence)");  
    }  
} else {  
    Serial.println("Inference error");  
}  
  
esp_camera_fb_return(fb);  
delay(1000);  
}
```



Literature Cited

- [1] Edge Impulse Inc., “*Edge Impulse Documentation – Image Classification, Deployment, and Arduino Integration,*” [Online]. Available: <https://docs.edgeimpulse.com>
- [2] Arduino, “*Arduino IDE – Official Documentation and Downloads,*” [Online]. Available: <https://www.arduino.cc/en/software>
- [3] Adafruit Industries, “*Using SH1106 OLED Displays with Arduino,*” [Online]. Available: <https://learn.adafruit.com/monochrome-oled-breakouts/overview>
- [4] Espressif Systems, “*ESP32-CAM Development Board Datasheet,*” [Online]. Available: <https://www.espressif.com/en/products/devkits/esp32-cam/overview>
- [5] M. A. Hossain, M. A. Khan and M. Hasan, “*Vision-Based Sign Language Recognition: A Review,*” in *International Journal of Computer Applications*, vol. 177, no. 12, pp. 12–19, Nov. 2019.
- [6] A. Jain, “*Sign Language Recognition Using Edge Machine Learning and Microcontrollers,*” in *International Journal of Embedded Systems and AI*, vol. 3, no. 2, pp. 45–50, 2022.



Acknowledgements

We would like to express our heartfelt gratitude to all those who contributed to the successful completion of this project. We would like to thank our project guide, **Rupin Pranay Navyakranth**, for their invaluable guidance, support, and constant encouragement throughout the course of this work. Their technical expertise and insightful feedback were instrumental in shaping the project to its current form. We also deeply thankful to the faculty members of the **NIT Warangal**, for providing the necessary resources and a conducive learning environment during the course of this project. We sincerely thank our peers and friends for their cooperation, constructive suggestions, and moral support. Their involvement and teamwork helped us overcome technical challenges during various stages of the implementation. Lastly, we would like to extend our special thanks to our families for their unwavering support, patience, and motivation, which kept us focused and determined throughout the project.